UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA DE SOFTWARE

Ingeniería de Requisitos

Sílabo y material de estudio

Código de la asignatura: **ISR-401**

Cuarto nivel — 3 créditos — 96 horas presenciales

*Material de estudio elaborado conforme al estándar internacional **IEEE/ISO/IEC 29148:2018**, la guía **SWEBOK v4.0 (2024)**, los modelos de calidad **ISO/IEC 25010:2023** y la segunda edición del manual de Pohl (Springer, 2024).*

Período académico: 2025 – 2026

Quevedo – Los Ríos – Ecuador

«La especificación de requisitos es el momento de la ingeniería del software en el que se decide, en última instancia, si el sistema va a resolver el problema correcto. Todo lo que viene después es construcción; lo que viene antes es ingeniería de requisitos. »

— *adaptado de Klaus Pohl [**Pohl2024**]*

Aviso: Este material acompaña al *Programa Analítico* oficial de la asignatura aprobado por la Facultad de Ciencias de la Computación. Los contenidos están enmarcados en los estándares y guías internacionales que se citan a lo largo del texto y se sustentan en bibliografía verificable, según las normas IEEE.

Índice general

Índice de figuras

Índice de tablas

Semana 1

Proceso de ingeniería de requisitos y contexto del sistema

Un proceso de Ingeniería de Requisitos sin un marco que lo organice se reduce a entrevistas dispersas y documentos inconexos; un marco sin proceso es solo teoría. La Ingeniería de Requisitos madura cuando ambos confluyen en un ciclo repetible, medible y mejorable.

adaptado de **Pohl2024**

Objetivos de aprendizaje de la semana

Al finalizar esta segunda semana, el estudiante estará en capacidad de:

1. Comprender la naturaleza *procesual* de la Ingeniería de Requisitos (IR), identificando sus actividades, artefactos y actores [**Pohl2024**, **SWEBOK_v4_2024**].
2. Comparar los **tres grandes modelos de proceso** de IR (cascada, iterativo y ágil), reconociendo sus ventajas, desventajas y contextos de aplicación [**Sommerville2016**, **Inayat2015**, **Curcio2018**].
3. Articular el proceso de IR con el **ciclo de vida del software** normalizado por *ISO/IEC/IEEE 12207:2017* [**ISO12207_2017**].
4. Aplicar criterios de **calidad y madurez** al proceso de IR, conforme al modelo Requirements Engineering Process Maturity Model (REPM) y a los principios de *ISO/IEC 33001:2015* [**Sawyer1997**, **ISO33001_2015**, **Niazi2005**].
5. Operar el **framework de IR de Pohl**: tres actividades core, dos transversales y tres tipos de artefactos [**Pohl2024**].
6. Distinguir los principales **tipos de sistemas de software** –información, embebidos, software-intensivos y autoadaptativos– y sus consecuencias para la IR [**Cheng2009**, **Yang2014**, **Pohl2024**].
7. Establecer correctamente el **límite del sistema** y formular una **visión** de proyecto trazable y comunicable [**Pohl2024**, **Wiegers2013**].
8. Analizar el contexto de un sistema mediante las **ocho perspectivas** de Pohl: uso, sujeto, TI, desarrollo, negocio, innovación, sociedad y técnica [**Pohl2024**].

1.1 Tema 3. Proceso de requisitos

1.1.1 Subtema 3.1. Modelos de proceso de requisitos. Actores

¿Por qué hablamos de «proceso»de IR?

La palabra **proceso** no es decorativa: cuando hablamos de «proceso de IR»estamos afirmando que las actividades de elicitar, documentar, negociar, validar y gestionar requisitos *no se realizan por casualidad ni en cualquier orden*, sino que constituyen un **flujo ordenado, repetible y mejorable** cuya ejecución puede medirse y comparase entre proyectos [**Sommerville2016**, **Pohl2024**, **SWEBOK_v4_2024**].

Definición

Proceso de Ingeniería de Requisitos. Conjunto estructurado de actividades –y de las prácticas y artefactos asociados– que transforman las necesidades de los Stakeholders en requisitos especificados, acordados, validados y mantenidos a lo largo del ciclo de vida del sistema [**Pohl2024**, **Kotonya1998**, **SWEBOK_v4_2024**].

Pensar en proceso aporta tres ventajas concretas: (i) permite **repetir** buenos resultados de un proyecto a otro, (ii) habilita la **medición y mejora** del trabajo y (iii) hace posible **comparar y certificar** a los equipos contra estándares como Capability Maturity Model Integration (CMMI) o *ISO/IEC 33001:2015* [**CMMI_v3_2023**, **ISO33001_2015**, **Niazi2005**].

Los tres grandes modelos de proceso de IR

La literatura distingue, de forma estable y consensuada, tres grandes familias de modelos de proceso de IR: el modelo **en cascada (lineal)**, el modelo **iterativo** y el modelo **ágil** [**Sommerville2016**, **Pressman2015**, **Pohl2024**, **Curcio2018**]. La figura **??** sintetiza visualmente sus diferencias.

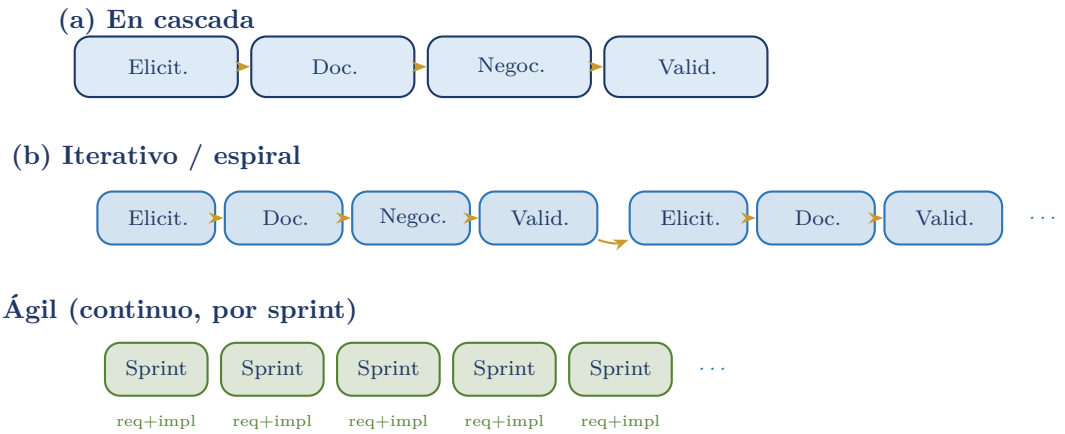


Figura 1.1. *Tres modelos de proceso de IR. (a) En cascada: actividades ejecutadas una vez en secuencia. (b) Iterativo: ciclos sucesivos refinan el conjunto de requisitos. (c) Ágil: requisitos descubiertos y desarrollados sprint a sprint. Adaptado de [***Royce1970***, **Boehm1988**, **Sommerville2016**, **Cao2008***].*

Modelo en cascada (lineal). Inspirado en el ciclo de vida clásico propuesto por Royce [**Royce1970**], organiza la IR como una secuencia de fases que se ejecutan *una sola vez* y producen un Especificación de Requisitos de Software (ERS) casi definitivo antes de pasar al diseño. Es el modelo más antiguo y aún sigue vigente en proyectos altamente regulados (defensa, banca central, sistemas médicos críticos), donde un cambio tardío resulta inaceptable [**Sommerville2016**, **Pressman2015**, **Hofmann2001**].

Modelo iterativo. Reconoce que la IR no se cierra de un solo intento. La idea, que Boehm formalizó con su modelo en espiral [**Boehm1988**], es producir una versión preliminar del ERS, validarla con Stakeholders, detectar omisiones y ambigüedades, y **volver al ciclo** para refinarla. Cada iteración disminuye la incertidumbre. Suele combinarse con prototipos.

Modelo ágil. Inspirado en el *Manifesto for Agile Software Development* [**Beck2001**], propone descubrir y elaborar los requisitos de forma *continua*, dentro de cada *sprint*, junto con el código y las pruebas. Las historias de usuario reemplazan a los párrafos formales, y el Product backlog sustituye al ERS monolítico [**Cohn2004**, **Schwaber2020**, **Cao2008**, **Inayat2015**, **Curcio2018**].

Cuadro comparativo de los tres modelos

Tabla 1.1. *Comparación de los tres modelos de proceso de IR según varios autores [***Sommerville2016***,* **Pressman2015***,* **Pohl2024***,* **Inayat2015***,* **Curcio2018***].*

Dimensión	Cascada	Iterativo	Ágil
Documento principal	ERS extenso, único.	ERS evolutivo en versiones.	Backlog vivo de historias.
Comunicación stakeholder	Inicio y aprobación final.	Al cierre de cada iteración.	Continua, diaria, en sprint.
Cambios	Costosos, vía CCB.	Asumibles entre iteraciones.	Esperados, parte del proceso.
Riesgo de descubrimiento tardío	Alto.	Medio.	Bajo.
Trazabilidad	Estricta y formal.	Formal por iteración.	Más débil; depende de herramienta.
Ideal para	Sistemas críticos, regulados.	Proyectos medianos, sistemas con incertidumbre acotada.	Web/móvil, productos de mercado.
Ventajas	Predictible; auditable; contractual.	Equilibrio entre rigor y flexibilidad.	Rápido, adaptable, centrado en valor.
Desventajas	Rígido; costoso de cambiar; supone conocimiento total.	Más complejo de planificar.	Riesgo de *deuda de requisitos*; trazabilidad débil.

Idea clave. Los tres modelos no son mutuamente excluyentes: *coexisten* en proyectos reales. La banca ecuatoriana, por ejemplo, suele aplicar cascada para módulos centrales (core bancario, contabilidad) y ágil para los canales digitales y aplicaciones móviles [**Mendez2017**, **Curcio2018**].

Actores del proceso de IR

Las normas [**IEEE29148_2018**] y [**ISO12207_2017**] definen un conjunto mínimo de roles que participan en el proceso. Pohl los presenta de manera operativa [**Pohl2024**]: el **ingeniero de requisitos** (analista) es el rol central, pero opera en una red de actores con responsabilidades complementarias.

Tabla 1.2. *Roles típicos en el proceso de IR.*

Rol	Responsabilidad principal
Patrocinador (sponsor)	Provee recursos y respaldo institucional. Aprueba alcance y baselines.
Stakeholder primario	Beneficiario directo del sistema; aporta requisitos y los valida.
Stakeholder secundario	Afectado indirecto (auditores, reguladores). Aporta restricciones.
Ingeniero de requisitos / analista	Conduce el proceso; elicita, documenta, negocia y mantiene el ERS.
Arquitecto / diseñador	Recibe el ERS; aporta retroalimentación de factibilidad técnica.
Tester	Deriva pruebas a partir de los requisitos; participa en su verificación.
Product Owner (ágil)	Custodio del backlog; prioriza historias y representa al cliente.
Comité de Control de Cambios (CCB)	Aprueba o rechaza cambios al ERS o al backlog congelado.

Ejemplo en Ecuador

Caso: equipo del proyecto integrador del semestre. En el proyecto integrador de la asignatura, el equipo de 3–5 estudiantes asume los roles centrales: un **analista líder** (responsable del ERS), un **modelador** (diagramas UML), un **documentador** (control de versiones del ERS) y un **enlace con el cliente real** (un Gobierno Autónomo Descentralizado (GAD) local, una microempresa de Quevedo, una clínica del cantón). El profesor actúa como **patrocinador y CCB**; el cliente real cumple el rol de **stakeholder primario**. Esta distribución refleja, a escala, la estructura típica de un proyecto profesional.

1.1.2 Subtema 3.2. Administración y soporte de procesos. Interrelación con el ciclo de vida del software

Administración del proceso de IR

Un proceso de IR no se ejecuta solo: requiere ser *planificado, asignado, monitoreado y ajustado*. Pohl distingue tres dimensiones de administración [**Pohl2024**]:

1. **Administración del contexto**: reconocer y documentar el contexto del sistema (las ocho perspectivas que veremos en el Tema 4) y mantenerlo actualizado.
2. **Administración de las actividades core**: planificar la elicitación, la documentación y la negociación, asignar responsables y establecer hitos.

3. **Administración de los artefactos**: controlar versiones, configuraciones y baselines del ERS y de sus modelos asociados.

Soporte: técnicas, herramientas y plantillas

El soporte al proceso se concreta en **tres recursos** principales que el equipo debe organizar antes de empezar [**Pohl2024**, **Wiegers2013**]:

- **Técnicas estandarizadas**: entrevistas, talleres, observación, prototipos, modelado UML, Easy Approach to Requirements Syntax (EARS), Must-have, Should-have, Could-have, Won't-have (técnica de priorización) (MoSCoW), Kano, Fagan, walkthroughs.
- **Herramientas Computer-Aided Software Engineering (CASE)**: Jira, Trello, Enterprise Architect, Sparx EA, Figma, draw.io, Confluence, GitLab Issues, ReqIF para intercambio.
- **Plantillas y guías**: plantilla IEEE 29148:2018 para el ERS, plantillas EARS para requisitos, plantilla del documento de visión, plantillas de matrices de trazabilidad.

Interrelación con el ciclo de vida del software

La IR no es una etapa aislada del ciclo de vida del software: *atraviesa todo el ciclo*, desde la concepción hasta el retiro. La norma *ISO/IEC/IEEE 12207:2017* define los procesos del ciclo de vida del software y ubica a la IR explícitamente como un proceso técnico clave [**ISO12207_2017**]. La figura **??** muestra cómo la IR se relaciona con las demás actividades.

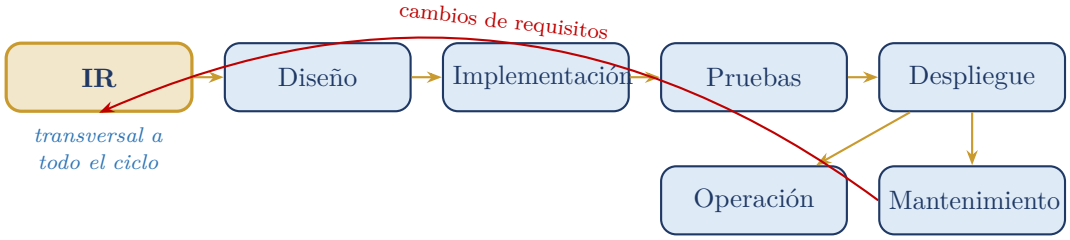


Figura 1.2. *La IR en el ciclo de vida del software. Adaptado de [***ISO12207_2017***,* **SWEBOK_v4_2024***,* **Pohl2024***].*

Tres ideas clave se desprenden [**ISO12207_2017**, **Sommerville2016**]:

- La IR **precede y condiciona** todo lo que viene después: si un requisito está mal, el diseño, el código y la prueba heredan el error.
- La IR **recibe retroalimentación** continua: durante el diseño se descubren ambigüedades; durante las pruebas, omisiones; durante la operación, requisitos emergentes.
- Por eso la IR se considera **transversal y dinámica**, no una fase cerrada.

Ejemplo en Ecuador

Caso: ciclo de vida del sistema de matrícula Universidad Técnica Estatal de Quevedo (UTEQ). Cuando la Secretaría Nacional de Educación Superior, Ciencia, Tecnología e Innovación (SENESCYT) emite una nueva resolución de homologación de asignaturas, el sistema universitario debe ajustarse. Esto inicia un *ciclo de retroalimentación* hacia la IR: se identifican nuevos requisitos, se actualiza el ERS, se redice riseñan los módulos y se prueban. Sin un proceso de IR establecido, este cambio se «parcha»en código y termina degradando la mantenibilidad del sistema [**Mendez2017**, **Bjarnason2014**].

1.1.3 Subtema 3.3. Perfeccionamiento y calidad del proceso de IR

¿Por qué medir y mejorar el proceso?

El estudio *Naming the Pain in Requirements Engineering* (Naming the Pain in Requirements Engineering (NaPiRE)) [**Mendez2017**], basado en encuestas a más de 200 empresas en 10 países, identificó que las cuatro causas más frecuentes de problemas en proyectos de software son *deficiencias del proceso de IR*: comunicación inadecuada con stakeholders, requisitos incompletos, requisitos cambiantes y especificaciones ambiguas. Esto motivó toda una corriente de modelos de **perfeccionamiento del proceso** [**Sawyer1997**, **Niazi2005**, **Hofmann2001**].

Modelos de madurez aplicados a IR

Existen al menos tres marcos de referencia complementarios:

1. CMMI (Capability Maturity Model Integration). Modelo general de madurez de procesos software cuyo nivel de capacidad se evalúa de 1 (inicial) a 5 (en optimización) [**CMMI_v3_2023**]. La IR aparece principalmente en el área de proceso «Requirements Development»y «Requirements Management».

2. *ISO/IEC 33001:2015*. Define el marco de evaluación de procesos software como sucesor de ISO/IEC 15504 (SPICE) [**ISO33001_2015**]. Establece atributos de proceso (process attributes) y niveles de capacidad de proceso de 0 a 5.

3. REPM (Requirements Engineering Process Maturity). Propuesto por Sawyer, Sommerville y Viller [**Sawyer1997**], es un modelo *específico para IR* que organiza las prácticas en tres niveles según se introduzcan en la organización (tabla **??**).

Tabla 1.3. *Modelo REPM de Sawyer, Sommerville y Viller* [**Sawyer1997**].

Nivel	Carácter	Prácticas típicas
1. Inicial	Caótico, dependiente del individuo.	No hay procedimientos definidos; los analistas hacen «lo que pueden».
2. Repetible	Prácticas básicas adoptadas.	Estructura del ERS, identificación de stakeholders, control de versiones, definición de plantillas.
3. Definido	Prácticas avanzadas e integradas.	Trazabilidad de extremo a extremo, análisis formal, métricas, mejora continua del proceso.

Métricas para evaluar la calidad del proceso de IR

Para «perfeccionar»el proceso es indispensable medirlo. Las métricas más usadas en literatura y práctica son [**Hofmann2001**, **Niazi2005**, **Mendez2017**, **Bjarnason2014**]:

- **Volatilidad de requisitos**: cantidad de cambios al ERS después de su línea base, normalizado por mes o por sprint.
- **Cobertura de trazabilidad**: porcentaje de requisitos con trazabilidad bidireccional documentada.
- **Densidad de defectos detectados en revisión de ERS**: número de defectos por página de ERS detectados en inspección.
- **Tiempo medio de resolución de un cambio de requisito**: desde su solicitud hasta su aprobación o rechazo formal.
- **Satisfacción de los stakeholders**: encuesta tras la validación o tras una iteración.

Advertencia / Error frecuente

Cuidado con medir lo fácil y olvidar lo importante. Una organización puede tener números brillantes en «cantidad de requisitos documentados»y, simultáneamente, fracasar en producción por requisitos no relevantes. Las métricas del proceso de IR deben mirar *calidad*, no solo *cantidad*: requisitos completos, verificables y trazables [**Mendez2017**].

1.1.4 Subtema 3.4. El framework de IR: actividades core, actividades transversales y artefactos

Estructura del framework de Pohl 2024

La segunda edición del libro de Pohl (2024) presenta un framework de IR compuesto por cuatro componentes que se relacionan entre sí [**Pohl2024**]:

1. **El contexto del sistema**, descrito mediante *ocho perspectivas* (uso, sujeto, TI, desarrollo, negocio, innovación, sociedad y técnica). Es el «marco»dentro del cual ocurre todo.
2. **Tres actividades core**: elicitación, documentación, y negociación / resolución de conflictos.
3. **Dos actividades transversales (cross-sectional)**: validación y gestión.
4. **Tres tipos de artefactos de requisitos**: metas, escenarios y requisitos orientados a

solución.

La figura **??** muestra el framework con sus relaciones. (Una versión simplificada se introdujo en la Semana 1; aquí se presenta con todos los elementos.)

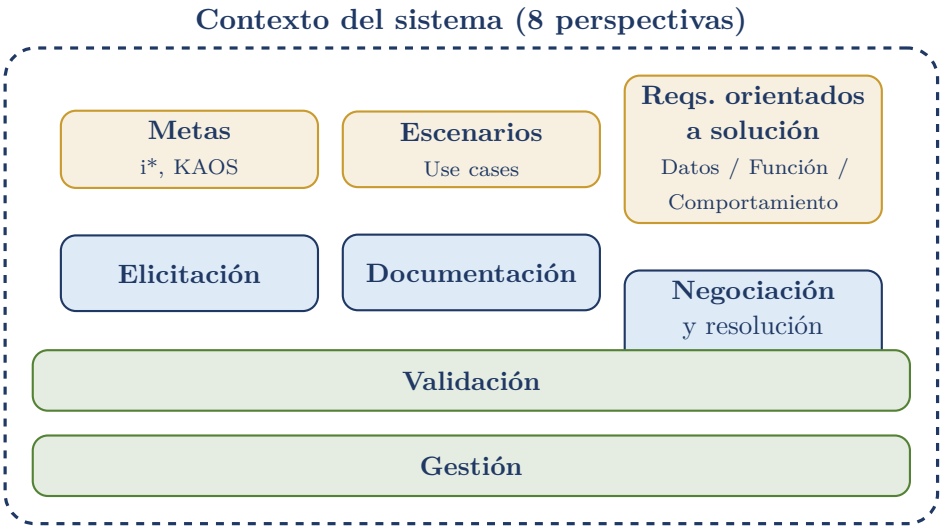


Figura 1.3. *Framework completo de IR de Pohl 2024: contexto, tres artefactos, tres actividades core y dos actividades transversales [***Pohl2024***].*

Las tres actividades core

1. Elicitación de requisitos. Actividad de descubrir, identificar y obtener requisitos a partir de las fuentes (stakeholders, documentos, sistemas existentes, regulaciones, contexto). Se profundizará en la Unidad 2, Semanas 4–6.

2. Documentación de requisitos. Actividad de registrar los requisitos elicitados de forma adecuada (lenguaje natural, modelos, plantillas estructuradas). Se profundizará en la Unidad 3, Semanas 7–10.

3. Negociación y resolución de conflictos. Actividad de *acordar* entre los stakeholders qué requisitos son los correctos, dadas las restricciones del proyecto. Pohl da en su edición 2024 mucho más peso a esta actividad que en la primera, porque es donde se generan los conflictos clave [**Pohl2024**].

Las dos actividades transversales

1. Validación de requisitos. Confirma que los requisitos representan correctamente las necesidades de los stakeholders y son adecuados para servir de base al desarrollo. Se profundizará en la Unidad 4, Semanas 11–13.

2. Gestión de requisitos. Mantiene el conjunto de requisitos consistente y actualizado a lo largo del ciclo de vida: trazabilidad, control de cambios, baselines, priorización dinámica.

Lo importante es que ambas son *transversales*: se ejecutan **en paralelo** a las tres actividades core, no después de ellas [**Pohl2024**]. Cada vez que se elicita o se documenta un requisito, inmediatamente se valida y se gestiona.

Los tres tipos de artefactos

1. Metas (goals). Objetivos prescriptivos de alto nivel que el sistema o la organización deben alcanzar. Se descomponen jerárquicamente con operadores AND/OR. Se modelan en lenguajes como i* 2.0 y Knowledge Acquisition in autOmated Specification (KAOS) [**Yu1997**, **vanLamsweerde2001**].

2. Escenarios (scenarios). Descripciones concretas de secuencias de interacción entre actores y el sistema. Tipos: caso de uso, historia de usuario, escenario narrativo, *misuse case*. Se profundizan en la Unidad 2, Semana 6.

3. Requisitos orientados a solución. Documentan *qué* debe hacer el sistema desde tres perspectivas: **datos** (qué información maneja), **función** (qué procesa) y **comportamiento** (cómo reacciona en el tiempo). Se modelan con diagramas Entidad-Relación (modelo de datos) (ER), clases UML, Data Flow Diagram (Diagrama de Flujo de Datos) (DFD2), statecharts y máquinas de estado.

Ventaja del framework de Pohl. Sirve como una *lista de verificación arquitectónica del proyecto de IR*: al iniciar un proyecto, el equipo se pregunta para cada elemento (contexto, artefactos, actividades) ¿lo hemos cubierto? Esta sola pregunta evita la mayor parte de los olvidos típicos en IR [**Pohl2024**].

1.2 Tema 4. Contexto, tipos de sistemas y visión

1.2.1 Subtema 4.1. Tipos de sistemas y consecuencias para IR

Una clasificación práctica

No todos los sistemas de software son iguales: ignorar las diferencias es una causa frecuente de fracasos. Pohl 2024 [**Pohl2024**] y SWEBOK v4 [**SWEBOK_v4_2024**] distinguen al menos cuatro grandes tipos, que imponen retos distintos a la IR.

Tabla 1.4. *Tipos de sistemas y sus consecuencias para la IR.*

Tipo	Características	Reto principal para la IR
Sistemas de información (SI)	Soportan procesos administrativos; UI rica; bases de datos.	Comprender procesos de negocio; gestión del cambio organizacional.
Sistemas embebidos	Software dentro de un dispositivo; restricciones de tiempo, energía y memoria.	Requisitos no funcionales estrictos; integración HW/SW; certificaciones.
Sistemas software-intensivos	Software interactúa intensivamente con otros sistemas y dispositivos.	Volumen y complejidad; interfaces múltiples; cadena de proveedores.
Sistemas auto-adaptativos	Reconfiguran su comportamiento en runtime ante cambios del entorno.	Requisitos contextuales y adaptativos; incertidumbre; loop MAPE-K.

Sistemas de información

Son los sistemas más comunes en empresas, gobierno y academia [**Sommerville2016**]. Capturan, almacenan, procesan y entregan información a usuarios humanos. Ejemplos: Enterprise Resource Planning (ERP), Customer Relationship Management (CRM), sistemas académicos, facturación, contabilidad.

Consecuencias para la IR: el reto está en **entender el negocio**. Si el analista no comprende cómo trabaja Tesorería del Servicio de Rentas Internas (Ecuador) (SRI), los requisitos serán superficiales. La elicitación demanda mucha entrevista, observación y análisis documental [**Pohl2024**, **Wiegers2013**].

Ejemplo en Ecuador

Sistema de tesorería de un GAD cantonal. Un sistema que gestione cobros del GAD de Quevedo (impuestos prediales, patentes, permisos, tasas) es un sistema de información típico. La IR debe capturar reglas de cálculo de tasas, exenciones legales locales, formatos de comprobantes válidos para el SRI y procesos de conciliación bancaria. Si se omite alguno, el sistema falla en producción durante el cobro masivo.

Sistemas embebidos

Son sistemas software-hardware integrados en un dispositivo físico que controlan o supervisan procesos del mundo real con restricciones de tiempo, energía y memoria [**Pohl2024**, **SWEBOK_v4_2024**]. Ejemplos: Electronic Control Unit (Unidad de Control Electrónico) (ECU) automotriz, marcapasos, contador de luz inteligente, controlador industrial.

Consecuencias para la IR: dominio de los Non-Functional Requirement (sinónimo en inglés de RNF) (NFR); las certificaciones (*Safety* de ISO 26262 en automoción, IEC 62304 en dispositivos médicos) generan requisitos adicionales muy específicos. El ERS suele incluir requisitos de tiempo de respuesta en milisegundos y de energía en miliamperios.

Ejemplo en Ecuador

Empacadora bananera automatizada. En las empacadoras de banano de Los Ríos hay sistemas embebidos que controlan cintas, brazos y balanzas con tolerancias estrictas (peso ±50 g, sincronización de cintas en ±10 ms). La IR debe especificar esos umbrales con precisión y referenciar los reglamentos de exportación, porque un error aritmético se traduce en pérdida económica directa.

Sistemas software-intensivos

Son sistemas en los que el software interactúa intensivamente con otros sistemas, dispositivos y personas [**Cheng2009**]. Una banca móvil moderna, un sistema hospitalario integrado o un «smart city»son ejemplos. La complejidad surge no del software individual, sino de las *interacciones* entre numerosos componentes.

Consecuencias para la IR: aumento del volumen de interfaces; cadena extensa de proveedores; alta incertidumbre. Se requieren *requisitos de integración*, *requisitos de operación distribuida* y mucha trazabilidad.

Sistemas autoadaptativos

Son sistemas capaces de **monitorear su entorno y modificar su propio comportamiento** en runtime para mantener sus metas frente a cambios o incertidumbre [**Cheng2009**, **Yang2014**]. La arquitectura canónica es **MAPE-K**: Monitor, Analyze, Plan, Execute, sobre una base de Knowledge compartida. Ejemplos: vehículos autónomos, aplicaciones de «smart vehicles», balanceadores en la nube, asistentes basados en Large Language Model (Modelo Grande de Lenguaje)s (LLMs).

Consecuencias para la IR: aparece una nueva categoría de requisitos –los **requisitos contextuales y adaptativos**– que describen condiciones del entorno y reglas de adaptación [**Yang2014**, **Cheng2009**]. Tradicionalmente, los requisitos se asumían *conocidos al inicio*; en sistemas adaptativos hay que asumir que *surgirán durante la operación*. Esto exige nuevos enfoques como los modelos KAOS extendidos o las *contextual goal models*.

Tendencia actual (2023–2025). La IR para sistemas adaptativos –especialmente para sistemas con componentes de LLM y de aprendizaje automático– es una de las áreas de investigación más activas del momento, con avances importantes documentados en [**Ferrari2025**, **Cheng2024**, **Arora2024**, **Necula2024**, **Pujiharto2024**].

1.2.2 Subtema 4.2. Límite del sistema, descomposición y visión

El límite del sistema

Definición

Límite del sistema (system boundary). Frontera conceptual que separa lo que pertenece al sistema a construir (*el sistema*) de lo que pertenece a su entorno (el contexto). Definirlo correctamente es una de las primeras y más críticas decisiones de la IR [**Pohl2024**, **IEEE29148_2018**, **INCOSE_SEH_2023**].

Un límite mal definido produce dos errores opuestos pero igualmente costosos:

- **Sub-especificación**: dejar fuera del sistema componentes que en realidad le competen (por ejemplo, omitir el módulo de firma electrónica en un sistema de comprobantes electrónicos del SRI).
- **Sobre-especificación**: incluir como propios componentes que pertenecen al entorno (por ejemplo, especificar el motor de bases de datos de Oracle dentro del sistema, en lugar de considerarlo una restricción de plataforma).

La figura **??** muestra los tres elementos que la IR debe distinguir desde el inicio: el sistema, el contexto relevante y el entorno externo [**Pohl2024**].

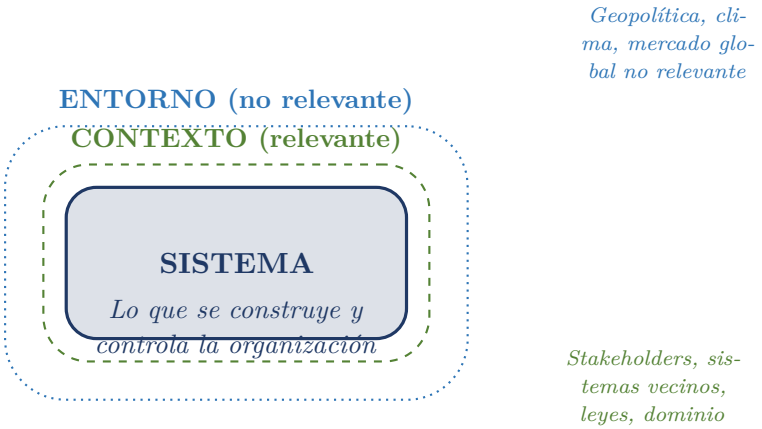


Figura 1.4. *Sistema, contexto y entorno: los tres niveles que la IR debe distinguir. Adaptado de [**Pohl2024**].*

Descomposición del sistema

Una vez establecido el límite, la IR suele descomponer el sistema en *subsistemas* y *componentes lógicos*, cada uno con sus requisitos asignados [**Pohl2024**, **INCOSE_SEH_2023**, **IEEE29148_2018**]. Esta descomposición se documenta en el ERS o, mejor, en el documento de arquitectura preliminar.

Ejemplo en Ecuador

Sistema de salud pública del Ministerio de Salud Pública del Ecuador (MSP). Un sistema integrado del MSP para gestión de turnos, recetas e historia clínica suele descomponerse en subsistemas: gestión de turnos, gestión de pacientes, historia clínica electrónica, recetas, integración con farmacias y analítica. La IR debe asignar a cada subsistema sus requisitos y gestionar las interfaces entre ellos.

La visión del sistema

Definición

Visión del sistema. Descripción concisa, motivadora y compartida de *lo que el sistema debe llegar a ser y del valor que aportará*. Suele documentarse en una o dos páginas y guía las decisiones a lo largo del proyecto [**Pohl2024**, **Wiegers2013**].

Wiegers & Beatty proponen una plantilla muy difundida en la industria para el documento de visión [**Wiegers2013**], sintetizada en la tabla **??**.

Tabla 1.5. *Plantilla de documento de visión adaptada de [***Wiegers2013***].*

Sección	Contenido orientativo
1. Oportunidad de negocio	¿Qué problema o necesidad existe?
2. Declaración del problema	¿A quién afecta y cuál es su impacto?
3. Declaración de la posición	Frase de una línea: «Para X, que requiere Y, el sistema Z hace W».
4. Stakeholders y usuarios	Quiénes son y qué les interesa.
5. Beneficios principales	Tres a cinco beneficios cuantificables.
6. Funcionalidades de alto nivel	Lista corta de capacidades clave.
7. Restricciones y supuestos	Tecnológicas, legales, de tiempo, económicas.
8. Criterios de éxito	Cómo se sabrá que el proyecto fue exitoso.

Ejemplo en Ecuador

Plantilla de visión: sistema de matrícula UTEQ.

Para los estudiantes y la Secretaría Académica de la UTEQ, que requieren un proceso de matrícula rápido, transparente y trazable durante los períodos críticos del calendario académico, el Sistema Integrado de Matrícula «MatrículaUTEQ»permitirá inscribirse, consultar prerrequisitos, generar órdenes de pago y obtener el comprobante oficial en menos de cinco minutos por estudiante, soportando 1 500 inscripciones concurrentes en hora pico.

Esta única frase orienta todo el proyecto: público, dolor a resolver, solución y atributos clave.

1.2.3 Subtema 4.3. Las ocho perspectivas de contexto de Pohl

¿Por qué ocho perspectivas?

En su primera edición, Pohl introdujo *cuatro* facetas de contexto. La segunda edición de 2024 [**Pohl2024**] las expandió a **ocho perspectivas**, reflejando que el contexto de los sistemas modernos –autoadaptativos, distribuidos, alimentados por datos– es mucho más rico que hace una década. Cada perspectiva aporta información necesaria para construir requisitos completos.

La figura **??** representa las ocho perspectivas en torno al sistema.

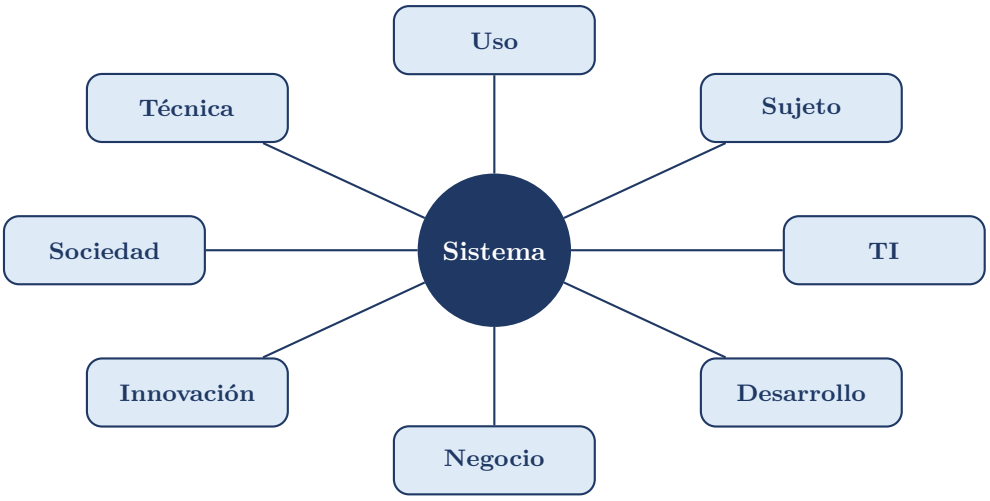


Figura 1.5. *Las ocho perspectivas de contexto de Pohl 2024 [***Pohl2024***].*

Detalle de cada perspectiva con ejemplos ecuatorianos

Tabla 1.6. *Las ocho perspectivas de contexto, sus preguntas guía y ejemplos.*

Perspectiva	Pregunta guía	Ejemplo (sistema de turnos del)
1. Uso	¿Quiénes lo usan, en qué tareas, en qué condiciones?	Pacientes con diferente nivel de alfabetización digital, médicos en consulta, personal administrativo.
2. Sujeto	¿Sobre qué entidades del mundo opera?	Pacientes, citas, tratamientos, recetas, médicos.
3. TI	¿Con qué sistemas de TI debe interactuar?	Historia clínica, sistema de farmacia, identificación digital.
4. Desarrollo	¿En qué proceso, con qué metodología, con qué equipo se construye?	Equipo distribuido, metodología ágil, despliegue continuo.
5. Negocio	¿Qué procesos de negocio soporta? ¿Qué políticas?	Política de turnos del MSP, atención emergente vs. programada, copagos.
6. Innovación	¿Qué oportunidades de innovación hay?	Predicción de no-asistencia con IA; turnos por canal de mensajería.
7. Sociedad	¿Qué impacto social tiene? ¿Qué normas?	Igualdad de acceso en zonas rurales; protección de datos según LOPDP.
8. Técnica	¿Qué restricciones técnicas hay? (HW, red, etc.)	Conectividad limitada en hospitales rurales; equipos heterogéneos.

Perspectiva de uso. Caracteriza a los usuarios y sus tareas. Recoge perfiles, nivel de alfabetización, frecuencia de uso, contextos de uso (oficina, calle, operación quirúrgica). Es la fuente directa de requisitos de *Interaction capability* (ISO 25010:2023) [**ISO25010_2023**].

Perspectiva del sujeto. Caracteriza el dominio del problema: las entidades del mundo real sobre las que opera el sistema. En un sistema bancario son cuentas, transacciones, clientes, divisas. Esta perspectiva alimenta los *modelos de datos* y diagramas ER.

Perspectiva TI. Caracteriza los sistemas existentes con los que debe interoperar. En un proyecto público ecuatoriano, esto incluye típicamente el ESIGEF (Ministerio de Finanzas), el portal del Servicio Nacional de Contratación Pública (SERCOP), el sistema del SRI, el del Instituto Ecuatoriano de Seguridad Social (IESS), y la Plataforma Gubernamental.

Perspectiva de desarrollo. Caracteriza el proceso, la metodología y el equipo: si será cascada o ágil, si el equipo es distribuido o local, si hay subcontratación, qué herramientas se usarán. Influye en los requisitos de proceso (Semana 1, Subtema 2.1).

Perspectiva de negocio. Caracteriza los procesos de negocio que el sistema debe soportar y las políticas que debe respetar. En la UTEQ, por ejemplo: política de matrículas, política de exámenes recuperatorios, reglamento de homologación.

Perspectiva de innovación. «¿Qué podríamos hacer diferente o mejor con este sistema?». Es la perspectiva más prospectiva: identifica oportunidades de aplicar nuevas tecnologías (IA, IoT, blockchain) o nuevos modelos operativos. En la edición 2024 de Pohl tiene mayor protagonismo [**Pohl2024**].

Perspectiva de sociedad. Caracteriza el impacto social, ético y legal del sistema. Cubre accesibilidad, equidad, sostenibilidad, protección de datos (LOPDP, GAD), inclusión cultural y lingüística [**Penzenstadler2014**, **Ecuador_LOPDP_2021**]. Es donde aparecen los *stakeholders indirectos*.

Perspectiva técnica. Caracteriza las restricciones tecnológicas: red, hardware, sistemas operativos, plataformas en la nube, dispositivos. En zonas rurales del Ecuador, esta perspectiva impone con frecuencia restricciones de conectividad y de potencia que no aparecen en proyectos urbanos.

Cómo aplicar las ocho perspectivas en la práctica

Pohl recomienda usar las perspectivas como un **checklist estructurado** al inicio del proyecto [**Pohl2024**]: para cada una se agendan al menos una entrevista o un taller con stakeholders relevantes y se documentan los hallazgos. La tabla **??** muestra una plantilla práctica.

Tabla 1.7. *Plantilla de análisis de contexto por perspectivas.*

Perspectiva	Stakeholder a entrevistar	Hallazgos clave	Requisitos derivados (preliminares)
Uso	Usuarios finales	...	RF/RNF de Interaction capability
Sujeto	Experto del dominio	...	Modelo conceptual de datos
TI	Arquitecto / DBA	...	RF de integración / restricciones
Desarrollo	Project Manager / scrum master	...	Requisitos de proceso
Negocio	Product Owner / dueño del proceso	...	Reglas de negocio, RF
Innovación	Líderes de área	...	Oportunidades de mejora
Sociedad	Compliance, legal, RR.HH.	...	Restricciones legales, accesibilidad
Técnica	Operaciones / DevOps	...	Restricciones técnicas, RNF de plataforma

Ejemplo en Ecuador

Aplicación práctica al proyecto integrador del semestre. Cada equipo del curso tomará su sistema objetivo (un GAD cantonal, una microempresa de Quevedo, una clínica privada, un sistema agroindustrial) y completará la plantilla anterior con al menos un hallazgo concreto y un requisito preliminar por perspectiva. El resultado se presentará en el primer hito del proyecto integrador.

1.3 Mapa síntesis de la Semana 2

La tabla **??** conecta los conceptos vistos durante la semana y muestra dónde aparecerán de nuevo a lo largo del semestre.

Tabla 1.8. *Mapa de cierre de la Semana 2.*

Pilar	Aporte conceptual	Dónde se profundiza
Modelos de proceso (3.1)	Cascada, iterativo, ágil; actores.	Unidad 5 (proyecto integrador).
Administración / ciclo (3.2)	La IR como proceso transversal del ciclo de vida.	Unidad 4 (gestión).
Calidad y madurez (3.3)	CMMI, ISO 33001, REPM, métricas de proceso.	Unidad 4 (gestión, métricas).
Framework de Pohl (3.4)	Tres core, dos transversales, tres artefactos.	Unidades 2, 3 y 4 (todo el resto del curso).
Tipos de sistemas (4.1)	SI, embebidos, software-intensivos, autoadaptativos.	Unidad 2 (elicitación contextual).
Límite y visión (4.2)	Sistema vs. contexto vs. entorno; doc. de visión.	Unidad 5 (kickoff del proyecto).
Ocho perspectivas (4.3)	Checklist sistemático para análisis de contexto.	Unidad 2 (elicitación) y todo el ERS.

1.4 Práctica experimental y trabajo autónomo

Práctica / Actividad

Práctica experimental n.º 2 (en clase, 2 horas).

Objetivo: aplicar el framework de Pohl 2024 y las ocho perspectivas de contexto a un caso real ecuatoriano.

Caso: la UTEQ desea modernizar el sistema de matrícula en línea para soportar matrículas concurrentes durante la semana pico.

Tareas en equipos de 3–4 estudiantes:

1. Decidir y argumentar qué *modelo de proceso* es más apropiado (cascada, iterativo, ágil) y por qué.
2. Identificar los actores del proceso (al menos 6) con su rol principal.
3. Para cada una de las ocho perspectivas de contexto, identificar un stakeholder a entrevistar y al menos un requisito preliminar que se derivaría.
4. Redactar el documento de visión usando la plantilla de Wiegers [**Wiegers2013**], no más de una página.
5. Discutir en plenaria los hallazgos.

Práctica / Actividad

Tarea autónoma n.º 2 (4 horas, individual).

Caso: usted ha sido contratado como analista para un proyecto de un GAD parroquial rural del Ecuador que desea construir una *aplicación móvil* para reportar incidencias urbanas (baches, luminarias, basura).

Tareas:

1. Clasifique el sistema según los tipos de la sección 4.1 (información, embebido, software-intensivo, autoadaptativo) y argumente la clasificación.
2. Defina explícitamente el límite del sistema y diga qué queda dentro y qué queda fuera.
3. Llene la plantilla de las ocho perspectivas de contexto con al menos un hallazgo y un requisito preliminar por perspectiva.
4. Redacte el documento de visión (una página).
5. Identifique al menos dos riesgos del proceso de IR para este caso y proponga cómo gestionarlos.

Entregable: documento PDF de máximo 8 páginas. La rúbrica ponderará especialmente la calidad del análisis de contexto.

Práctica / Actividad

Lectura previa para la Semana 3 (aula invertida).

- Pohl, K. *Requirements Engineering*, 2.ª ed., Parte II completa (capítulos sobre contexto y perspectivas) [**Pohl2024**].
- Méndez Fernández et al. (2017), *Naming the pain in requirements engineering* [**Mendez2017**].
- Inayat et al. (2015), *Systematic literature review on agile requirements engineering practices and challenges* [**Inayat2015**].

1.5 Resumen ejecutivo de la semana

- La IR es un **proceso**: una secuencia ordenada, repetible y mejorable de actividades, no un conjunto de tareas improvisadas [**Pohl2024**, **SWEBOK_v4_2024**].
- Existen **tres modelos** de proceso de IR: en cascada, iterativo y ágil. No son excluyentes; coexisten en proyectos reales según el contexto [**Sommerville2016**, **Cao2008**, **Curcio2018**].
- El proceso de IR se administra en tres dimensiones: *contexto*, *actividades core* y *artefactos* [**Pohl2024**].
- La IR es **transversal** al ciclo de vida del software normalizado por *ISO/IEC/IEEE 12207:2017*: precede al diseño, recibe retroalimentación de las pruebas y absorbe los cambios del mantenimiento [**ISO12207_2017**].
- Modelos de madurez como CMMI, *ISO/IEC 33001* y REPM permiten medir y mejorar el proceso de IR [**CMMI_v3_2023**, **ISO33001_2015**, **Sawyer1997**, **Niazi2005**].
- El **framework de Pohl 2024** estructura la IR en cuatro componentes: contexto, tres

actividades core (elicitación, documentación, negociación), dos transversales (validación, gestión) y tres artefactos (metas, escenarios, requisitos orientados a solución) [**Pohl2024**].

- Existen al menos cuatro **tipos de sistemas** con consecuencias distintas para la IR: información, embebidos, software-intensivos y autoadaptativos [**Cheng2009**, **Yang2014**, **Pohl2024**].
- Establecer el **límite del sistema** (sistema vs. contexto vs. entorno) es una de las primeras decisiones críticas de la IR [**Pohl2024**, **INCOSE_SEH_2023**].
- El **documento de visión** (una a dos páginas) condensa el propósito del sistema en términos de problema, solución y beneficios [**Wiegers2013**].
- Las **ocho perspectivas de contexto** de Pohl 2024 (uso, sujeto, TI, desarrollo, negocio, innovación, sociedad, técnica) son un checklist sistemático para construir requisitos completos [**Pohl2024**].